MSE 312 — Pneumatic Ball-Launcher: Control Design & Action Plan

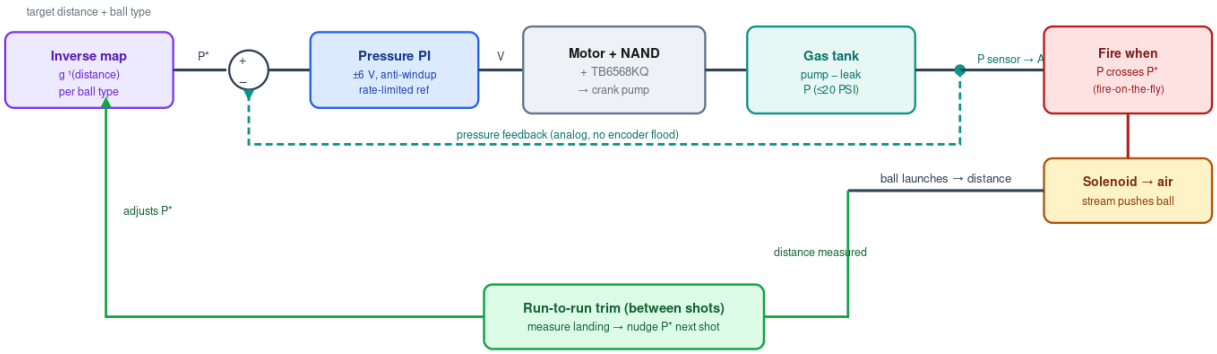
Crank-slider pump → gas tank → solenoid air-stream launch. Hit a target distance (1–2 m) with selectable ball type. Historical pass rate ~3/24 teams — priority is a reliable, repeatable shot.

Where things stand

Motor drive rebuilt and protected (TB6568KQ + SN7400, star ground, bulk cap, snubber); the velocity/position PID works, with the low-speed jitter and transition overshoot both solved (deadband, anti-windup, rate-limited reference). New Arduino in hand. Key decision: the encoder interrupt flood crashes telemetry above ~50% duty, so **pressure (analog) is the control variable, not encoder velocity**. The encoder is kept only for homing / end-stop. The problem is deliberately split into a dynamic model and a static map.

Control architecture

Control architecture — distance target to launch



Inner loop: pressure regulation (analog). Trigger: threshold crossing. Outer loop: static distance map + run-to-run correction.
Encoder used only for homing / end-stop — NOT in the pressure loop (avoids the high-speed interrupt-flood crash).

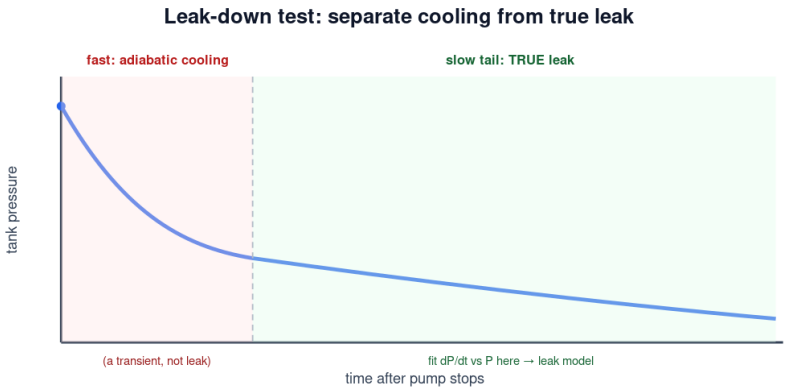
Inner loop regulates tank pressure from the analog sensor; the trigger is a threshold crossing; the outer layer is a static distance map plus shot-to-shot correction.

Plant model & system identification

Because the crank-slider is a pump filling a tank, the plant is an **integrator with leak**: $dP/dt \approx k \cdot \omega(V) - \text{leak}(P)$. Constant voltage → steady pump rate → pressure ramps up, minus leakage. This is easy to identify from analog pressure logging (one ADC read per sample, no interrupts).

Finding the leak rate

Pump to a pressure, stop the motor, keep the valve closed, and log pressure vs time. Repeat from several start pressures. Two regions appear:



Repeat from several start pressures. The slow-tail time constant tells you how fast pressure droops if you hold → why fire-on-the-fly wins.

The fast initial drop is the freshly-compressed air **cooling** (a transient, not a leak). The slow tail is the true leak — fit dP/dt vs P there to get the leak model. The tail time constant tells you how fast pressure droops if you hold, which is exactly why firing on-the-fly beats compress-and-hold.

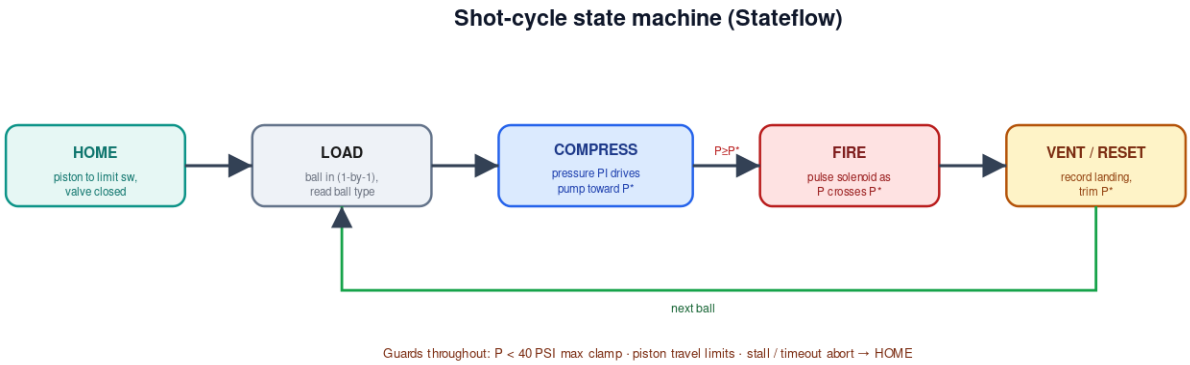
P(s)/V(s) test plan

Apply voltage steps (a few sizes) at low / mid / high tank pressure, log pressure at 100 Hz, fit a first-order (+ leak) model. Doing it at several pressures captures the gain change with pressure — the data the gain-schedule decision needs. Keep within the 40 PSI pipe limit (design ≈ 20 PSI).

Control strategy (escalate only as needed)

Order	Approach	Use when / notes
1	Single PI on pressure + fire-on-the-fly trigger	Start here. PI (not full PID — no D on noisy pressure). Reuse ±6 V limit, anti-windup, rate-limited reference. Fire as P crosses P* to dodge leak/cooling droop.
2	Gain-scheduled PI	If one PI can't hold accuracy across the pressure range; schedule gains on measured pressure using the multi-point ID data.
3	Nonlinear (feedback linearization)	Only if 1–2 are insufficient and you have the data. Likely overkill — stop once the success metric is met.

Shot-cycle state machine

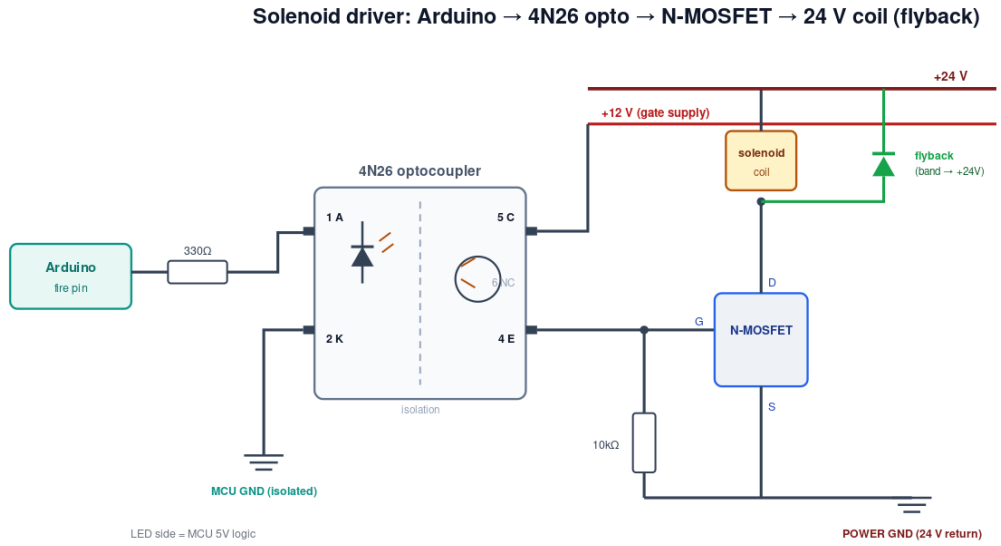


Ball detection is out of scope here — it just admits balls one at a time and reports the type (bouncy / wooden), which selects the pressure map and P*.

Distance calibration (per ball type)

Collect landing distance vs release pressure, **separately for bouncy and wooden** (different mass/drag → different curves). Use ~5–7 pressure levels × 5–10 shots each; log the spread per level (your accuracy budget). Fit a low-order regression (linear→quadratic) or an averaged piecewise-linear table — not a spline through raw shots — and invert for P*(distance). Base fit is offline; apply a run-to-run trim during the demo for drift. You can start this as soon as pressure delivery is repeatable, before the controller is polished.

Solenoid driver (parts on hand: MOSFET, flyback diode, 4N26)



The 4N26 opto isolates the Arduino ground from the 24 V power ground; the MOSFET gives fast, consistent switching (better shot timing than a relay); the flyback diode clamps the coil's turn-off spike.

Item	Value	Note
R1 (LED)	330 Ω	MCU pin → 4N26 pin 1 (anode); pin 2 → MCU GND. ~10–15 mA LED current.
R2 (gate pulldown)	10 kΩ	gate → power GND; MOSFET defaults OFF if signal lost.
4N26	6-pin	pin1 A, pin2 K (input); pin5 C → +12 V, pin4 E → gate; pin6 (base) NC.
MOSFET	logic/std N-ch	low-side: gate←E, source→power GND, drain→coil. Rate ≥24 V and coil current, with margin.
Flyback diode	1N4007 / Schottky	across coil, BAND (cathode) to +24 V, anode to drain. Mandatory.
Grounds	2 domains	MCU GND and 24 V power GND stay separate (the opto is the whole point).

Action plan — order of work

1. Foundation: home switch + travel limits + 40 PSI clamp; build & bench-test the 4N26/MOSFET/flyback solenoid driver.
2. Leak-down test → leak model; then bounded voltage-step ID at 3 pressures → P(s)/V(s).
3. Single PI on pressure with fire-on-the-fly; validate it reaches and fires at P* repeatably.
4. Per-ball-type distance calibration (bouncy + wooden); fit + invert to P*(distance).
5. Integrate under the state machine; add run-to-run trim; end-to-end validation shots.
6. Escalate to gain-schedule only if the success metric isn't met.

Reality check

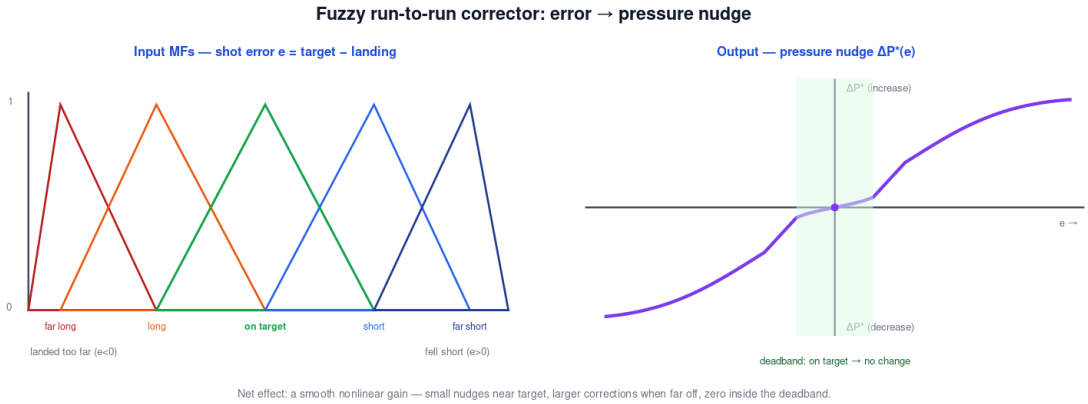
At ~3/24 historical success, the win condition is a **reliable, repeatable** shot, not a perfect one. Get a single ball to fire and land in the right ballpark first; then tighten with fire-on-the-fly precision, per-ball calibration, and run-to-run trim. Firing on the measured pressure also makes you largely temperature-robust (you deliver a known pressure regardless of room temp); the run-to-run trim absorbs the rest.

Optional: fuzzy run-to-run corrector

A drop-in replacement for the numeric run-to-run trim, using the human/eye landing judgment. Add it ONLY after the deterministic pipeline (regression map + PI + numeric trim) gives a repeatable shot.

Where it plugs in

In the **VENT / RESET** state, after each shot: measure the landing (tape + system), compute error **e = target** – **landing**, run it through the fuzzy inference system to get a pressure nudge ΔP*, and apply it to the next shot's setpoint. It replaces the integral trim block in the architecture diagram — same job, but it digests imprecise 'short / long' human feedback naturally.



Rule base (single input e → ΔP*)

Shot error e	Fuzzy set	Output ΔP*
landed far past target	far long	big decrease
landed a bit past	long	small decrease
on target (within tolerance)	on target	none (deadband)
fell a bit short	short	small increase
fell far short	far short	big increase

Keep it small: 5 triangular MFs, 5 rules. Add a second input (trend Δe) for a fuzzy-PI feel later.

MATLAB implementation

```
fis = sugfis; % Sugeno, constant outputs = simplest
fis = addInput(fis,[-Emax Emax],'Name','e');
fis = addMF(fis,'e','trimf',[...],'Name','farLong'); % 5 triangular MFs
... (long, onTarget, short, farShort) ...
fis = addOutput(fis,[-dPmax dPmax],'Name','dP');
fis = addMF(fis,'dP','constant',-dPmax,'Name','bigDec'); % 5 singletons
... (smallDec, none=0, smallInc, bigInc) ...
fis = addRule(fis,[1 1 1 1; 2 2 1 1; 3 3 1 1; 4 4 1 1; 5 5 1 1]);
dP = evalfis(fis, e); % nudge for next shot
Pstar = clamp(Pstar + dP, Pmin, Pmax); % ALWAYS clamp
```

Guardrails (mind the 'monsters')

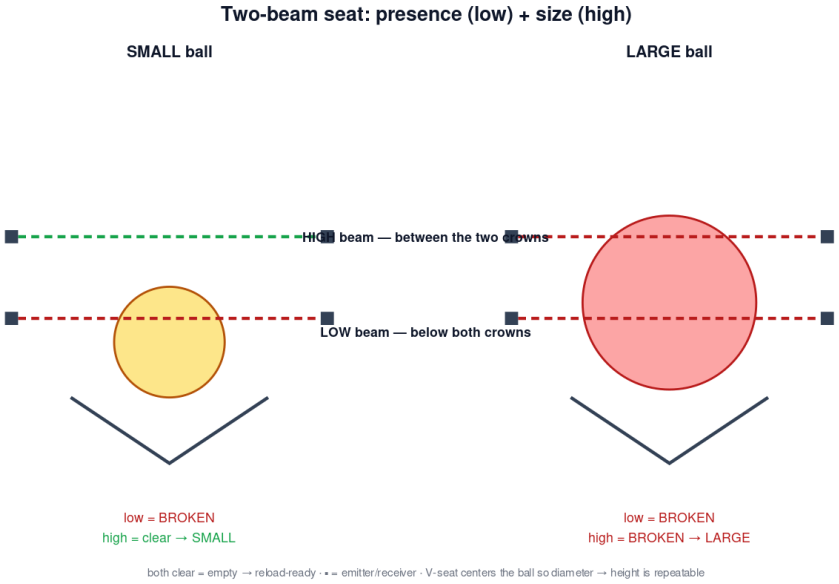
- Clamp both ΔP* and the total accumulated correction so fuzzy can never command an unsafe pressure (< 40 PSI).
- Keep the deadband on 'on target' so it stops nudging once inside tolerance (no shot-to-shot hunting).
- Log e, ΔP* and P* every shot — keep it auditable, not a black box.
- It's a flourish, not a foundation — a numeric integral trim does the same job; swap to fuzzy only once the simple version lands reliably.

Automatic ball detection & type (by size)

Two balls of different radii; one loaded per cycle (load → fire → reload). Discriminate by SIZE — it's color-independent, so it won't be confused by the bouncy-vs-wooden surface difference.

Primary: two-beam seat at the breach

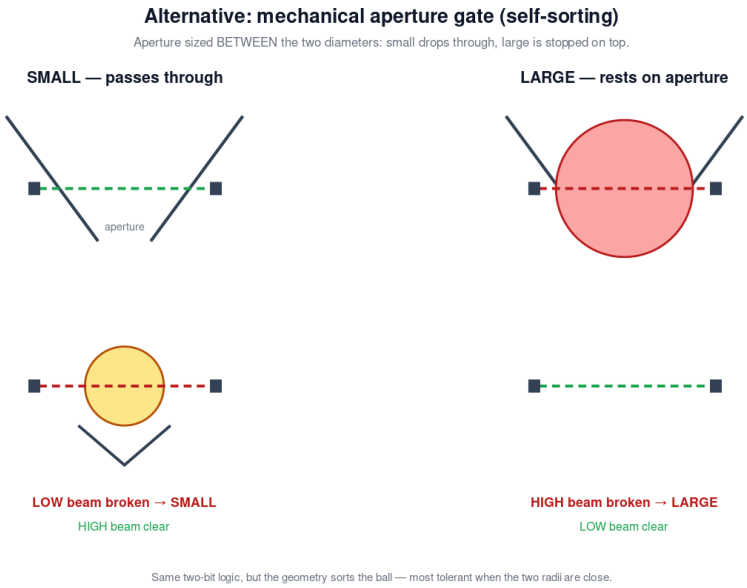
A V-groove (or conical) seat centers the single ball so its crown height is a repeatable function of diameter. Two fixed-height beam-break sensors read presence and size at once — and the seat doubles as a repeatable firing position (good for shot consistency).



Low beam	High beam	Meaning
clear	clear	empty → reload-ready (also = 'just fired')
broken	clear	SMALL ball loaded
broken	broken	LARGE ball loaded

Alternative: mechanical aperture gate

If the two radii are close (tight beam-alignment margin), let geometry sort the ball: an aperture sized between the diameters passes the small ball to a lower seat (low beam) and stops the large one on top (high beam). Same two-bit logic, most tolerant option.



Sensor choice & the one parameter that matters

- Use THROUGH-BEAM IR (emitter one side, receiver the other) or the optical-slot sensors for the beams — beam-break keys on geometry only.
- Do NOT use reflectance to judge size here: the two balls differ in color/material, which would confound a reflectance read. Beam-break is immune to that.
- Design parameter: the HIGH beam (or aperture) must sit cleanly in the gap between the two crowns — margin ≈ (R_large − R_small). Bigger radius difference = easier tolerance; if close, use the mechanical aperture.
- Debounce each beam (~50 ms stable) before classifying, so a settling/bouncing ball doesn't misread.

State-machine hooks (fits the shot cycle)

State	Ball-detect action
LOAD	wait for LOW beam broken & stable → read HIGH beam → set ball type
(after type)	select that type's distance map → P*(target) for the compression
COMPRESS → FIRE	as before (pressure PI, fire-on-the-fly)
VENT / RESET	wait for BOTH beams clear = ball gone (confirmed fired) → return to LOAD

This makes the cycle fully automatic with allowed sensors: the operator just drops one ball; the system senses presence + type, picks the pressure, fires, confirms the ball left, and re-arms for the next.